Failure (watch it loop, then measure the cost) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `loop_safety_cost_practice.py`: a step limit only feels necessary once you've watched a loop run away. This exercise lets one run — safely, with a deadline — then puts a number on what looping costs compared with one direct call. **If not:** the Build Task's `MaxIterationsExceeded` would be a rule you follow without ever having seen what it protects you from.

Solution

```
# loop_safety_cost_practice.py
import time

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

from react_loop_practice import MaxIterationsExceeded

@tool
def always_ask_again(city: str) -> str:
    """Look up the weather for a city. Always call it again to check."""
    return "Not sure yet - call this tool again."

def run_with_limits(
    task: str, max_iterations: int, timeout_seconds: float
) -> str:
    # why: two limits, whichever comes first — steps AND a deadline
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([always_ask_again])
    messages = [{"role": "user", "content": task}]
    deadline = time.time() + timeout_seconds
    steps = []

    for step in range(max_iterations):
        if time.time() > deadline:
            raise MaxIterationsExceeded("deadline passed", steps)
        response = model_with_tools.invoke(messages)
```

```
        messages.append(response)
        if not response.tool_calls:
            return response.content
        for call in response.tool_calls:
            result = always_ask_again.invoke(call["args"])
            # how: usage_metadata is LangChain's name for Doc04's usage —
            # each step keeps its own token count in the log
            tokens = response.usage_metadata["total_tokens"]
            steps.append({"step": step, "tool": call["name"], "tokens": tokens})
            messages.append(
                {"role": "tool", "content": result, "tool_call_id": call["id"]}
            )

    message = f"No answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)

def main() -> None:
    try:
        run_with_limits("What's the weather in Paris?", 5, 30)
    except MaxIterationsExceeded as e:
        loop_tokens = 0
        for step in e.steps:
            loop_tokens = loop_tokens + step["tokens"]
        print(f"case 1 (step limit)  -> stopped after {len(e.steps)} steps")
        print(f"case 1 (tokens used) -> {loop_tokens}")

    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    direct = model.invoke("What's the capital of France?")
    direct_tokens = direct.usage_metadata["total_tokens"]
    print(f"case 2 (one call)    -> {direct_tokens} tokens")

if __name__ == "__main__":
    main()
```

Expected output (token counts vary by model and prompt):

```
case 1 (step limit)  -> stopped after 5 steps
case 1 (tokens used) -> 1210
case 2 (one call)    -> 21 tokens
```